does 2 crore package is possible in software Engineer profile, if yes what are the stuffs I should prepare and what companies offer such packages, any real like placements samples, What kind of question they ask, give me sample questions as well

Short answer: **Yes, a ₹2 crore (CTC) package in software engineering is possible — but it's rare and only for top ~0.1% candidates.** Let's break it down clearly so you know what's real vs hype and how to actually aim for it.

🧠 1. Reality check: Is ₹2 crore possible?

✅ Yes, but under specific conditions:

- Mostly from:
  - **Quant / Trading firms** (highest chance)
  - **Top Big Tech (FAANG+) senior roles**
- Usually includes:
  - Base salary + bonus + **stock (RSU)**

👉 Real examples:

- IIT placements have seen **₹2–2.5 crore offers** (e.g., trading firms, global roles)

  The Times of India +1
- Some companies in India offer **₹1Cr–₹2Cr+ total compensation** for top engineers

  upGrad

⚠️ Important:

- These are **very few offers (1–5 students per campus)**
- Often require **exceptional skills + internships + elite colleges or equivalent skill level**

🏢 2. Companies that offer ₹1Cr–₹2Cr+

🔥 Highest paying (₹1–2Cr+)

- Quant firms:
  - Da Vinci, Optiver, Graviton, Quadeye, APT Portfolio
- Big Tech:

  - Google, Meta, Microsoft, Apple, Amazon upGrad
- Fintech / Hedge funds:
  - Goldman Sachs (senior roles), DE Shaw

🚀 High-paying startups & product companies

- Atlassian, Uber, Nvidia, Flipkart, Walmart Labs upGrad

💡 Insight:

- Quant firms pay highest because of **math + algorithms + low-latency systems**

📊 3. Real placement examples (India)

- IIT Kharagpur: **₹2.5 Cr highest package** Recruitment & P...
- IIT Hyderabad: **₹2.5 Cr offer (international trading firm)** The Times of India
- MNNIT: **₹74 LPA (DE Shaw)** The Times of India

👉 Notice pattern:

- Top college + strong DSA + internships = high package

📚 4. What you must prepare (MOST IMPORTANT)

Think of it in 4 layers:

🧩 1. Core (NON-NEGOTIABLE)

- Data Structures & Algorithms (DSA)
- Time complexity, optimization
- Problem solving (LeetCode level)

👉 Without this → no chance

⚙️ 2. Core CS subjects

- Operating Systems
- DBMS
- Computer Networks
- System Design (for higher roles)

💻 3. Development skills

At least ONE strong area:

- Backend (Java / Node / Go)
- Full stack (React + backend)
- OR Systems (C++ for low latency)

🤖 4. Advanced edge (this differentiates ₹20L vs ₹2Cr)

- Competitive Programming
- AI/ML / Distributed systems
- Low latency systems (for quant firms)

🎯 5. Interview process (what actually happens)

Round 1: Online coding

- 2–3 DSA problems (medium-hard)

Round 2–3: Coding interviews

- Deep problem solving
- Optimization

Round 4: System / CS round

- OS, DBMS, design

Final: Behavioral + resume deep dive

🧪 6. Sample questions (REAL STYLE)

🟢 Easy–Medium

- Two Sum
- Longest Substring Without Repeating Characters
- Merge Intervals

🔴 Hard (common in top companies)

- LRU Cache (design + code)
- Median of 2 sorted arrays
- Word Ladder

- Design Twitter / URL Shortener

⚡ Quant / Top-tier questions

- Optimize latency of a trading system
- Design a rate limiter
- Solve advanced DP / graph problems in 20–30 min

🧠 7. What TOP candidates do differently

This is the honest difference:

- Solve **500–1500+ DSA problems**
- Do **Codeforces / contests**
- Intern at:
  - Microsoft / Amazon / startups
- Build **real projects (not copy-paste)**
- Strong fundamentals (not just coding tricks)

🚀 8. Roadmap (simple and practical)

Year 1–2:

- Learn programming (C++/Java/Python)
- Start DSA

Year 2–3:

- Solve 300+ problems
- Build projects
- Start internships

Year 3–4:

- Advanced DSA + System design
- Mock interviews
- Apply to top companies

⚠️ Final truth (very important)

- ₹2 Cr is **not the goal — skill is**

- If you reach that level:
  - ₹30L–₹80L becomes **very achievable**
  - ₹1Cr+ comes with experience

💬 If you want

I can:

- Make a **personal roadmap for YOU (based on your current level)**
- Give **company-wise prep strategy (Google vs Amazon vs Quant)**
- Share **exact LeetCode list (top 100 questions)**

Just tell me your current year + skill level 👍

Sources

Give company-wise prep strategy (Google vs Amazon vs Quant)
Share exact LeetCode list (top 100 questions)

one by one

Perfect—this is exactly the right way to approach it. Let's start with **company-wise prep strategy** first. I'll keep it practical and honest so you know what actually works.

🏢 Company-wise Prep Strategy

🟦 1. Google (Most concept-heavy)

🎯 What Google looks for

- Deep understanding (not memorization)
- Clean, optimal code
- Strong fundamentals
- Ability to **explain clearly**

🧠 How to prepare

✅ DSA Focus

- Master:
  - Graphs (DFS, BFS, shortest path)
  - Dynamic Programming (very important)
  - Trees (LCA, traversals)
- Difficulty:
  - Mostly **Medium → Hard**

✅ Key mindset

Google cares about:

- "Why this solution?"
- "Can you optimize further?"
- "What are edge cases?"

👉 You must **talk through your thinking**

🧪 Example Google-style question

- "Given a grid, find shortest path with obstacles"
- "Design autocomplete system"
- "Longest increasing path in matrix"

⚠️ Common mistake

- Jumping to code without explaining logic → ❌ rejection

🧩 Extra edge

- Strong CS fundamentals (OS, DBMS basics)
- Clean coding style (very important)

🟧 2. Amazon (Most predictable & achievable)

🎯 What Amazon looks for

- Problem-solving speed
- Practical coding ability

- Behavioral fit (**Leadership Principles**)

🧠 How to prepare

✅ DSA Focus

- Arrays & Strings (VERY frequent)
- Hashing
- Sliding window
- Greedy

👉 Difficulty:

- Mostly **Easy** → **Medium (some Medium-Hard)**

✅ Behavioral (VERY IMPORTANT)

Amazon is different here.

You MUST prepare:

- "Tell me about a time when…"
- Leadership Principles:
  - Ownership
  - Customer obsession
  - Bias for action

👉 This round can reject strong coders.

🧪 Example Amazon-style question

- "Top K frequent elements"
- "Number of islands"
- "Longest substring without repeating characters"

⚠️ Common mistake

- Ignoring behavioral round → ❌ rejection

🧩 Extra edge

- Practice writing **bug-free code fast**

- Do mock behavioral interviews

🟥 3. Quant Firms (₹1Cr–₹2Cr tier)

This is a completely different game.

🎯 What they look for

- Extreme problem-solving ability
- Speed + accuracy under pressure
- Strong math + logic

🧠 How to prepare

✅ DSA Focus (Hard level)

- Advanced DP
- Graph theory
- Bit manipulation
- Probability / combinatorics

✅ Math + Logic

- Probability puzzles
- Expected value problems
- Brain teasers

✅ Language preference

- **C++ (important for performance)**

🧪 Example Quant questions

- "Expected number of coin tosses to get 3 heads in a row"
- "Design ultra-low latency system"
- "Solve this DP in 15 minutes"

⚠️ Reality check

- Questions are **harder than FAANG**
- Time pressure is intense

🧩 Extra edge

- Competitive programming (Codeforces)
- Solve 1000+ problems
- Strong mental math

⚖️ Comparison Summary

Feature	Google 🟦	Amazon 🟧	Quant 🟥
Difficulty	High	Medium	Very High
Focus	Concepts	Speed + behavior	Hardcore problem solving
DSA level	Medium–Hard	Easy–Medium	Hard
Extra skills	CS fundamentals	Leadership	Math + CP
Chances	Moderate	High	Very low (elite)

🚀 How YOU should choose

- Beginner → **Amazon path first**
- Intermediate → **Amazon + Google**
- Advanced → **Quant + Google**

👍 Next step

In the next message, I'll give you:

👉 **Top 100 LeetCode questions (carefully curated, not random)**
– grouped by topic
– exactly what top candidates solve

Just say **"next"** and I'll send it.

next

Great—this is the part that actually moves your level up if you do it seriously.

I'm giving you a **clean, high-quality Top 100 LeetCode list**, grouped by topic. This is **not random**—it's very close to what strong candidates actually solve for FAANG + quant prep.

🧠 TOP 100 LEETCODE LIST (CURATED)

🟢 1. Arrays & Hashing (Foundation – MUST MASTER)

1. Two Sum
2. Best Time to Buy and Sell Stock
3. Contains Duplicate
4. Product of Array Except Self
5. Maximum Subarray (Kadane)
6. Maximum Product Subarray
7. Find Minimum in Rotated Sorted Array
8. Search in Rotated Sorted Array
9. 3Sum
10. Container With Most Water

👉 If you're weak here → everything else becomes hard

🟡 2. Sliding Window

11. Longest Substring Without Repeating Characters
12. Longest Repeating Character Replacement
13. Minimum Window Substring 🔥
14. Permutation in String
15. Sliding Window Maximum

🔵 3. Two Pointers

16. Valid Palindrome
17. Two Sum II (sorted)
18. 3Sum (again—important)
19. Trapping Rain Water 🔥
20. Remove Nth Node From End

🌳 4. Trees

21. Maximum Depth of Binary Tree
22. Same Tree
23. Invert Binary Tree
24. Binary Tree Level Order Traversal
25. Lowest Common Ancestor (BST + Binary Tree) 🔥
26. Diameter of Binary Tree
27. Balanced Binary Tree
28. Subtree of Another Tree

🎄 5. Advanced Trees

29. Serialize and Deserialize Binary Tree 🔥
30. Kth Smallest Element in BST
31. Validate BST
32. Binary Tree Right Side View
33. Construct Binary Tree from Preorder & Inorder

🌐 6. Graphs

34. Number of Islands 🔥
35. Clone Graph
36. Pacific Atlantic Water Flow
37. Course Schedule 🔥
38. Course Schedule II
39. Graph Valid Tree
40. Word Ladder 🔥

🧮 7. Dynamic Programming (VERY IMPORTANT)

41. Climbing Stairs

42. House Robber
43. House Robber II
44. Longest Increasing Subsequence 🔥
45. Coin Change 🔥
46. Partition Equal Subset Sum
47. Longest Common Subsequence 🔥
48. Word Break
49. Combination Sum IV
50. Decode Ways 🔥

🔥 8. Backtracking

51. Subsets
52. Subsets II
53. Permutations
54. Combination Sum
55. Word Search 🔥

📚 9. Stack

56. Valid Parentheses
57. Min Stack
58. Evaluate Reverse Polish Notation
59. Daily Temperatures
60. Largest Rectangle in Histogram 🔥

🔗 10. Linked List

61. Reverse Linked List
62. Linked List Cycle
63. Merge Two Sorted Lists
64. Merge K Sorted Lists 🔥
65. Remove Nth Node From End
66. Copy List with Random Pointer 🔥

🧠 11. Binary Search

67. Binary Search

68. Search in Rotated Sorted Array
69. Find Peak Element
70. Median of Two Sorted Arrays 🔥

🧩 12. Heap / Priority Queue

71. Kth Largest Element in Array
72. Top K Frequent Elements 🔥
73. Find Median from Data Stream 🔥
74. Merge K Sorted Lists (again important)
75. Task Scheduler

⚡ 13. Greedy

76. Jump Game
77. Jump Game II
78. Gas Station
79. Hand of Straights

🧪 14. Intervals

80. Merge Intervals
81. Insert Interval
82. Non-overlapping Intervals 🔥
83. Meeting Rooms II

🧠 15. Advanced / Hard (Google + Quant)

84. LRU Cache 🔥
85. LFU Cache
86. Trapping Rain Water (again)
87. Word Ladder (again)
88. Minimum Window Substring (again)
89. Edit Distance 🔥
90. Regular Expression Matching

🧮 16. Math / Bit Manipulation

91. Single Number
92. Number of 1 Bits
93. Counting Bits
94. Reverse Bits
95. Missing Number

🧩 17. Bonus (Very High Value)

96. Design Twitter 🔥
97. Design Hit Counter
98. Random Pick with Weight
99. Alien Dictionary 🔥
100. Sliding Window Median 🔥

🎯 How to use this list (IMPORTANT)

Don't just "complete" it. Do this:

Step 1:

- Solve **topic-wise**, not randomly

Step 2:

For each problem:

- Try 30–40 min
- If stuck → learn → redo after 2 days

Step 3:

- Revise 🔁 (most people skip this → big mistake)

⏱️ Timeline

- Beginner: 2–3 months
- Serious prep: 1–2 months revision
- Total: ~4 months → strong level

⚠️ Honest advice

- 100 questions ≠ enough for quant
- But this list = **perfect for Amazon + Google base**

👍 If you want next

I can now give you:

- 📅 **Daily schedule (4–8 week plan)**
- 📊 **Beginner → Advanced roadmap**
- 🧪 **Mock interview questions with answers**

Just tell me what you want next.

Mock interview questions with answers

Love this step—this is where preparation turns into actual interview performance.

I'll simulate **real interview-style questions with answers + thinking process**, not just final code. Focus on *how to think and explain*, because that's what gets you hired.

🧪 MOCK INTERVIEW SET (with Answers)

🟢 1. Amazon-Level Question

❓ Question: Longest Substring Without Repeating Characters

Problem:
Given a string `s`, find the length of the longest substring without repeating characters.

🧠 How to think (say this in interview)

- Brute force → check all substrings → $O(n^2)$ ❌
- Better: Use **sliding window + set/map**
- Expand window until duplicate → shrink from left

✅ Optimal Solution (Sliding Window)

```python
def lengthOfLongestSubstring(s):
    char_set = set()
    left = 0
    max_length = 0

    for right in range(len(s)):
        while s[right] in char_set:
            char_set.remove(s[left])
            left += 1

    return max_length
```

Python

⏱️ Complexity

- Time: O(n)
- Space: O(n)

💬 What interviewer expects

- Clear explanation of window movement
- Handling duplicates properly

🟡 2. Google-Level Question

❓ Question: LRU Cache

Problem:

Design a data structure that supports:

- `get(key)`
- `put(key, value)`

  Both in **O(1)** time

🧠 Thinking process

- Need fast lookup → **HashMap**
- Need order tracking → **Doubly Linked List**
- Combine both

✅ Solution

```python
class Node:
    def __init__(self, key, value):
        self.key = key
        self.value = value
        self.prev = self.next = None

class LRUCache:
    def __init__(self, capacity):
        self.cap = capacity
        self.cache = {}

        self.left = Node(0, 0)   # LRU
        self.right = Node(0, 0)  # MRU

        self.left.next = self.right
```

‹/› Python

```python
    def remove(self, node):
        prev, nxt = node.prev, node.next
        prev.next, nxt.prev = nxt, prev

    def insert(self, node):
```

💬 What Google checks

- Can you **design + explain trade-offs**
- Why DLL over list?
- Why O(1)?

🔴 3. Hard Graph (Google / Quant)

❓ Question: Number of Islands

🧠 Thinking

- Each island = connected component
- Use **DFS or BFS**
- Mark visited cells

✅ Solution (DFS)

```python
def numIslands(grid):
    if not grid:
        return 0

    rows, cols = len(grid), len(grid[0])
    visited = set()

    def dfs(r, c):
        if (r < 0 or c < 0 or r >= rows or c >= cols or
            grid[r][c] == "0" or (r, c) in visited):
            return

        visited.add((r, c))

        dfs(r+1, c)
```

Python

```python
        dfs(r, c-1)

    count = 0

    for r in range(rows):
```

💬 Key explanation

- "Each DFS marks one island completely"

🟣 4. Dynamic Programming (Google Favorite)

❓ Question: Coin Change

🧠 Thinking

- Try all coins → minimize count
- DP[i] = min coins to make amount i

✅ Solution

```python
def coinChange(coins, amount):
    dp = [float('inf')] * (amount + 1)
    dp[0] = 0

    for i in range(1, amount + 1):
        for coin in coins:

    return dp[amount] if dp[amount] != float('inf') else -1
```

💬 What interviewer checks

- Can you build DP relation
- Can you explain transition clearly

🟥 5. Quant-Level (Thinking Heavy)

❓ Question: Expected tosses to get 2 consecutive heads

🧠 Thinking (this is what matters)

Let:

- E = expected tosses

Case breakdown:

- First toss = Head (½), then need 1 more head
- First toss = Tail (½), restart

Equation:

```
E = 1 + (1/2)*E1 + (1/2)*E
```

Solve recursively → Answer = **6**

💬 What they test

- Logical modeling
- Probability reasoning
- Not coding → thinking clarity

🎯 How to Practice Mock Interviews

Do this seriously:

Step 1:

Pick 1 question
→ Explain OUT LOUD (very important)

Step 2:

Follow this structure:

1. Clarify problem
2. Brute force
3. Optimize
4. Code
5. Edge cases

Step 3:

Time yourself (30–40 min)

⚠️ Biggest mistakes (I see all the time)

- Jumping to code ❌
- Not explaining ❌
- Not testing edge cases ❌
- Memorizing solutions ❌

🚀 If you want next level

I can:

- Take a **live mock interview with you**
- Give **company-specific mock (Amazon / Google / Quant)**
- Evaluate your answers like a real interviewer

Just say:
👉 "take my mock interview" or
👉 "give Amazon mock"

Explain me what is Quant in details, as a beginner how I can prepare and what I need to prepare, If I prepare for Quant, can I clear Amazon Google ? or any FAANG interviews

Great question—and honestly, this is where a lot of confusion happens. "Quant" sounds fancy, but it's very specific. Let's break it down clearly so you know what you're getting into.

🧠 What is a "Quant" role?

Quant = Quantitative Developer / Quantitative Researcher

These are engineers who work in **trading firms / hedge funds**, building systems that:

- Predict market movements
- Execute trades in microseconds
- Optimize profits using math + algorithms

💼 Types of Quant Roles

1. 👷‍♀️ Quant Developer (closest to SWE)

- Write high-performance code

- Work on trading systems, low latency
- Skills: **C++ + DSA + systems**

👉 This is what most CS students target

---

2. 📊 Quant Researcher

- Build mathematical models
- Heavy probability, statistics, ML

👉 Requires strong math (often beyond typical CS)

---

3. ⚡ Quant Trader

- Mix of math + decision making
- Fast thinking, probability intuition

🏢 Companies (Quant Firms)

- Graviton Research
- Quadeye
- Optiver
- Da Vinci
- APT Portfolio
- Tower Research
- Jane Street (global top tier)

👉 These are the ones giving ₹1Cr–₹2Cr packages

🧠 What makes Quant different from FAANG?

Area	FAANG (Google/Amazon)	Quant Firms
Coding	Medium–Hard DSA	Hard DSA (faster)
Math	Basic	Heavy (probability, puzzles)
Speed	Important	CRITICAL
Difficulty	High	VERY HIGH

Area	FAANG (Google/Amazon)	Quant Firms
Questions	Standard patterns	Unpredictable

📚 What YOU need to prepare (Beginner → Advanced)

I'll break it into layers so it's actionable.

🟢 1. Strong Coding Foundation (NON-NEGOTIABLE)

- Language:
  - **C++ (preferred)** or Python
- DSA:
  - Arrays, Strings, Hashing
  - Trees, Graphs
  - DP, Greedy

👉 Same base as FAANG, but deeper

🟡 2. Advanced Problem Solving

- Solve:
  - Hard LeetCode
  - Codeforces problems
- Topics:
  - Bit manipulation
  - Advanced DP
  - Graph optimization

🔵 3. Mathematics (THIS IS THE DIFFERENCE)

You must learn:

Probability

- Conditional probability
- Expected value
- Bayes theorem

Combinatorics

- Permutations & combinations
- Counting problems

Logic puzzles

- Brain teasers
- Pattern recognition

🔴 4. Speed + Thinking

Quant interviews test:

- Can you solve **hard problems fast (10–20 min)**
- Can you think under pressure

⚙️ 5. Systems (for Quant Dev)

- Low latency systems
- Memory optimization
- Multithreading basics

🧪 Example Quant Questions

These are real-style:

🧠 Probability

- Expected tosses to get 3 heads in a row
- Monty Hall problem

🧩 Logic

- 100 prisoners and 100 boxes
- Light bulb switching puzzles

💻 Coding

- Hard DP (optimized)
- Graph shortest path variations

- Bitmask problems

🚀 Beginner Roadmap (Step-by-step)

Month 1–2:

- Learn DSA basics
- Solve 150–200 problems

Month 3–4:

- Start **hard problems**
- Learn probability basics

Month 5–6:

- Competitive programming (Codeforces)
- Advanced math + puzzles

Month 6+:

- Mock interviews
- Focus on speed

❓ MOST IMPORTANT QUESTION

👉 If I prepare for Quant, can I crack FAANG?

✅ YES — and actually easier

If you genuinely reach quant level:

- Amazon → **easy**
- Google → **moderate**

Because:

- Quant prep = harder than FAANG

❗ BUT important nuance

If you ONLY do:

- Math puzzles
- Random hard problems

Then:

- You may FAIL FAANG (because they need structured DSA + communication)

🎯 Best Strategy (Smart approach)

👉 Don't choose one blindly

Ideal path:

1. Build **FAANG-level DSA first**
2. Then go into **Quant-level difficulty + math**

⚠️ Honest Reality

- Quant = top 0.1%
- Requires:
  - Consistency
  - Deep thinking
  - Not just "watching videos"

🧠 Simple analogy

- Amazon → marathon
- Google → chess
- Quant → speed chess + puzzles + math olympiad

👍 If you want next

I can:

- Give **Quant-specific problem list (must-do questions)**
- Create a **daily schedule combining FAANG + Quant**
- Take a **quant-style mock interview**

Just tell me 👍

Now explain me what is FAANG, what I should do to become pro in FAANG, what are the opportunities I can get if I become pro, what package I can expect and cover the syllabus

Alright—let's make FAANG very clear and practical, without hype.

🧠 What is FAANG?

FAANG = Facebook (Meta), Amazon, Apple, Netflix, Google

Today people often extend it to:

- **MAANG / FAANG+**
- Includes: Microsoft, Uber, Atlassian, Nvidia, etc.

👉 In simple terms:
FAANG = top product-based tech companies with high pay + strong engineering culture

---

💼 What does a FAANG Software Engineer do?

- Build large-scale systems (millions of users)
- Work on:
  - Backend systems
  - APIs
  - Distributed systems
  - UI / mobile apps

👉 Example:

- Google → Search, Maps
- Amazon → e-commerce backend
- Meta → Instagram, WhatsApp

🎯 What does “Pro in FAANG” mean?

Not just getting selected.

It means:

- Strong in **problem solving**
- Solid **CS fundamentals**
- Can design **real-world systems**
- Write **clean, scalable code**

📚 Complete FAANG Syllabus (THIS IS YOUR BLUEPRINT)

🟢 1. Data Structures & Algorithms (MOST IMPORTANT)

Must master:

- Arrays, Strings
- Hashing
- Linked List
- Stack, Queue
- Trees (Binary Tree, BST)
- Graphs
- Heap / Priority Queue

Algorithms:

- Binary Search
- Sliding Window
- Two Pointers
- Recursion + Backtracking
- Dynamic Programming 🔥
- Greedy
- Graph algorithms (BFS, DFS, Dijkstra)

👉 This alone = **60–70% of interview**

🟡 2. Core CS Subjects

Operating Systems

- Processes vs Threads
- Deadlocks
- Scheduling

DBMS

- SQL queries
- Indexing
- Normalization

Computer Networks

- HTTP/HTTPS
- TCP vs UDP
- Latency basics

👉 Especially important for **Google, Meta**

🔵 3. System Design (for higher roles / strong candidates)

- Design:
  - URL shortener
  - Instagram / WhatsApp
  - Rate limiter
- Concepts:
  - Load balancing
  - Caching
  - Database scaling

🟣 4. Development Skills

At least one:

- Backend (Java / Node / Go / Python)
- OR Full stack (React + backend)

👉 This helps in:

- Resume shortlisting
- Practical interviews

🔥 5. Behavioral (especially Amazon)

- Tell me about a challenge
- Conflict with team
- Leadership example

👉 Use **STAR method**

🚀 What YOU should do (step-by-step plan)

🥇 Phase 1: Foundation (0 → decent)

- Learn one language (Java / C++ / Python)
- Solve ~150 problems
- Understand basics of all topics

🥈 Phase 2: Strong DSA

- Solve 300–500 problems
- Focus on:
  - Patterns
  - Optimization
- Start mock interviews

🥉 Phase 3: Interview Ready

- Revise top 100 questions
- Practice:
  - System design basics
  - Behavioral answers

ChatGPT

🏁 Phase 4: Apply + Interview

- Referrals + LinkedIn
- Mock interviews
- Timed practice

💰 Salary (India & Global Reality)

IN India (Freshers / 0–2 yrs)

- Amazon: ₹20–40 LPA
- Google: ₹30–60 LPA
- Uber / Atlassian: ₹40–80 LPA

👉 Top cases: ₹1Cr+ (with stocks)

🌍 Abroad (US etc.)

- $120K – $200K base
- Total: $150K–$300K+

⚠️ Important:

- "CTC" includes stocks (not all cash)

🚀 Opportunities if you become FAANG-level

This is where it gets exciting:

💼 1. High-paying jobs

- Top product companies
- Startups (high ESOPs)

🌍 2. Global opportunities

- US / Europe jobs
- Remote high-paying roles

🧠 3. Career growth

- Senior engineer → Architect → Engineering manager

🚀 4. Startup / entrepreneurship

- Strong system knowledge
- Ability to build products

💸 5. Freelance / consulting

- High hourly rates

⚖️ FAANG vs Quant (quick clarity)

- FAANG → structured, scalable, achievable
- Quant → elite, math-heavy, very competitive

👉 FAANG is the **best first target**

⚠️ Biggest mistakes to avoid

- Only watching tutorials ❌
- Not practicing enough ❌
- Ignoring revision ❌
- Skipping behavioral prep ❌

🎯 Final simple roadmap

If I simplify everything:

1. DSA (daily)
2. Projects (1–2 solid)
3. Core CS (basics)
4. Mock interviews

👉 Do this consistently → FAANG is realistic

👍 If you want next

I can help you with:

- 🗓️ **Daily 3–6 month FAANG plan**
- 📊 **Your current level analysis**
- 🎯 **Company-specific roadmap (Amazon vs Google)**

Just tell me where you are right now (college year / skills), and I'll tailor it exactly for you.

as a beginner from IT backgroud and know programming basics, in how many months I can become pro level in FAANG

I'll give you a very honest answer—not motivational fluff, but what actually happens in real prep.

🧠 Short Answer

👉 **6–12 months** to reach *strong FAANG interview level*
👉 **12–18 months** to become *"pro level"*

📊 What "pro level" actually means

You're considered strong if you can:

- Solve **medium problems in 15–20 min**
- Solve **hard problems in 30–40 min**
- Explain clearly (this is HUGE)
- Handle system design basics
- Do interviews confidently (not just coding alone)

⏱️ Timeline Breakdown (for YOU)

Since you said:

beginner + IT background + know programming basics

You're in a **good starting position**.

🟢 Phase 1: Foundation (1–2 months)

Goal:

- Understand all DSA basics

You should cover:

- Arrays, Strings, Hashing
- Linked List, Stack, Queue
- Trees basics

👉 Solve ~100–150 problems

🟡 Phase 2: Strong DSA (2–4 months)

Goal:

- Become comfortable with patterns

Focus:

- Sliding window
- Binary search
- Graphs
- DP (start here)

👉 Solve ~200–300 problems

🔵 Phase 3: Advanced + Interview Prep (2–3 months)

Goal:

- Interview readiness

Focus:

- Hard problems
- System design basics
- Mock interviews

👉 Revise top 100 questions

🟣 Phase 4: Polishing (1–3 months)

Goal:

- Speed + confidence

Focus:

- Timed practice
- Weak areas
- Behavioral prep

🗓️ Total Time

Effort Level	Time Needed
2–3 hrs/day	10–12 months
4–6 hrs/day	6–8 months
Full-time grind	4–6 months

⚠️ Reality Check (important)

Most people fail because:

- They **don't stay consistent**
- They **don't revise**
- They just "watch solutions"

👉 The difference is NOT intelligence
👉 It's **daily consistency for months**

🎯 What YOU should aim for

Instead of "pro", aim for:

👉 "Interview-ready in 6 months"

Because once you reach that:

- You can crack:
  - Amazon ✅
  - Good product companies ✅
- Then upgrade to:

- Google / Meta level

🧠 Key Insight (very important)

You don't need:

- 1000 problems ❌
- Genius IQ ❌

You need:

- ~300–500 **quality problems**
- Deep understanding
- Revision

🚀 If you follow properly

In **6–8 months**, you can realistically reach:

- ₹20–40 LPA level companies
- Strong FAANG chances

In **12+ months**:

- ₹50L–₹1Cr+ potential (top tier)

👍 If you want next step

I can:

- Create a **daily schedule for you (based on your time availability)**
- Track your progress week-by-week
- Give **exact problems for each day**

Just tell me:
👉 How many hours per day you can study
👉 Your current level (very basic / solved some problems / etc.)

I'll design a plan just for you.

Thank you for clarification!

You're very welcome! 😊
You're asking the right questions—that already puts you ahead of a lot of people starting out.

If you stay consistent and follow a structured plan, this goal is absolutely achievable. It might feel overwhelming now, but once you start solving problems daily, things begin to click faster than you expect.

Whenever you're ready, I can help you with:

- a **daily study plan**
- **tracking your progress**
- or even a **mock interview**

Just drop a message anytime 👍

Give exact problems for each day - current level very basic - after 9.5 hours of office, hardly 2-3 hours

Got it—that's a tough schedule (9.5 hrs work + 2–3 hrs study), so the plan must be **realistic, focused, and not overwhelming**.

I'll give you a **day-by-day plan (first 4 weeks)**. After that, you'll repeat the pattern with higher difficulty.

👉 Goal: **Build strong DSA foundation without burnout**

🗓️ RULES (VERY IMPORTANT)

- ⏱️ Daily time: **2–3 hours**
- 📌 Per day:
  - 1 main problem (deep understanding)
  - 1 easy/related problem (speed + confidence)
- 🔁 Revise previous problems every 3rd day

🟢 WEEK 1 (Arrays + Basics)

Day 1

- Two Sum
- Contains Duplicate

Day 2

- Best Time to Buy and Sell Stock
- Maximum Subarray

Day 3

- Product of Array Except Self
- Revision (Day 1 problems)

Day 4

- Maximum Product Subarray
- Contains Duplicate II

Day 5

- Find Minimum in Rotated Sorted Array
- Revision (Day 2)

Day 6

- Search in Rotated Sorted Array
- Practice Two Sum again (fast)

Day 7 (Light Day 🔁)

- Revise ALL week problems
- Re-solve without seeing solution

🟡 WEEK 2 (Strings + Sliding Window)

Day 8

- Valid Palindrome
- Valid Anagram

Day 9

- Longest Substring Without Repeating Characters 🔥

Day 10

- Longest Repeating Character Replacement
- Revision (Day 8)

Day 11

- Minimum Window Substring (try hard, don't worry if stuck)

Day 12

- Permutation in String
- Revision (Day 9)

Day 13

- Group Anagrams
- Practice sliding window again

Day 14 (Revision Day)

- Revise all sliding window problems

🔵 WEEK 3 (Linked List + Stack)

Day 15

- Reverse Linked List
- Linked List Cycle

Day 16

- Merge Two Sorted Lists
- Valid Parentheses

Day 17

- Min Stack
- Revision (Day 15)

Day 18

- Remove Nth Node From End

Day 19

- Daily Temperatures
- Revision (Day 16)

Day 20

- Evaluate Reverse Polish Notation

Day 21 (Revision Day)

- Re-solve key linked list + stack problems

🌳 WEEK 4 (Trees – VERY IMPORTANT)

Day 22

- Maximum Depth of Binary Tree
- Same Tree

Day 23

- Invert Binary Tree
- Binary Tree Level Order Traversal

Day 24

- Diameter of Binary Tree
- Revision (Day 22)

Day 25

- Balanced Binary Tree

Day 26

- Lowest Common Ancestor 🔥

Day 27

- Subtree of Another Tree

Day 28 (BIG Revision)

- Revise ALL tree problems

🧠 HOW TO STUDY EACH DAY (CRITICAL)

For each problem:

Step 1 (10–15 min)

- Try yourself

Step 2 (20 min)

- If stuck → learn solution

Step 3 (20 min)

- Code it yourself (no copy)

Step 4 (5 min)

- Explain out loud

⚠️ KEY STRATEGY (this matters more than problems)

- Don't rush ❌
- Depth > number of problems ✅
- Revision = success 🔁

🎯 After 4 Weeks

You will:

- Understand core patterns
- Start feeling confident
- Be ready for:
  - Graphs
  - DP
  - Medium-level problems

💬 Honest expectation

With this pace (2–3 hrs daily):

- 1 month → foundation
- 3 months → strong
- 6 months → interview ready

👍 Next step (I recommend)

After you finish Week 1, come back and say:
👉 "next week plan"

I'll:

- Adjust based on your performance
- Increase difficulty gradually
- Keep it aligned with FAANG level

You’re doing this the right way—slow, consistent, structured. That’s exactly how people actually make it. 🚀